Sebastian Bordt

EBERHARD KARLS
UNIVERSITÄT
TÜBINGEN

Deep Learning

Exercise 2 – Image Classification

Release date: October 28, 2026

1 Pen and Paper

1.1 Analytic Derivative Calculation

Compute the derivatives of the following functions.

a) $\text{ReLU}_c(x) = \max(0, x - c)$, where $x \in \mathbb{R}$ and c is a scalar constant.

b) $\text{SiLU}(x) = x\,\sigma(x)$, where $x \in \mathbb{R}$ and $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid function. You may use that $\sigma'(x) = \sigma(x)\,(1 - \sigma(x))$.

c) $\text{Tanh}(x) = \frac{2}{1+e^{-2x}} - 1$, $x \in \mathbb{R}$

d) The softmax function maps a vector of scores $\mathbf{x} = (x_1, \ldots, x_n)^\top \in \mathbb{R}^n$ to a vector of probabilities $\text{softmax}(\mathbf{x}) \in \mathbb{R}^n$ with the elements

$$\text{softmax}(\mathbf{x})_j = \frac{\exp(x_j)}{\sum_{k=1}^n \exp(x_k)}, \qquad j = 1, \ldots, n.$$

Compute all partial derivatives $\partial\,\text{softmax}(\mathbf{x})_j / \partial x_k$ for $j, k \in \{1, \ldots, n\}$.

Hint: distinguish the cases $j = k$ and $j \neq k$.

e) Let $\mathbf{p} \in \mathbb{R}^n$ be a vector of predicted probabilities ($p_i > 0$ and $\sum_{i=1}^n p_i = 1$) and $\mathbf{y} \in \{0, 1\}^n$ a one-hot label vector (exactly one element is 1, all others are 0). The cross-entropy loss is $H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^n y_j \log p_j$, where log is the natural logarithm. Calculate its gradient wrt. every element of $\mathbf{p}$, i.e. $\frac{\partial H}{\partial p_i}(\mathbf{y}, \mathbf{p})$ for $i = 1, \ldots, n$.

f) Now combine d) and e). Let the predicted probabilities be the softmax of a score vector $\mathbf{s} \in \mathbb{R}^n$, i.e. $\mathbf{p} = \text{softmax}(\mathbf{s})$, and write the cross-entropy loss as a function of the scores, $\mathcal{L}(\mathbf{s}) = H\big(\mathbf{y}, \text{softmax}(\mathbf{s})\big)$. Show that

$$\frac{\partial \mathcal{L}}{\partial s_k} = p_k - y_k \qquad \text{for } k = 1, \ldots, n.$$

Hint: by the chain rule, $\frac{\partial \mathcal{L}}{\partial s_k} = \sum_{j=1}^n \frac{\partial H}{\partial p_j} \frac{\partial p_j}{\partial s_k}$.

1.2 Softmax invariance

Let softmax be the softmax function from 1.1 d), and let $\mathbf{x} + c = (x_1 + c, \ldots, x_n + c)^\top$ denote adding a scalar $c \in \mathbb{R}$ to every element of $\mathbf{x} \in \mathbb{R}^n$.

a) Prove that $\text{softmax}(\mathbf{x}) = \text{softmax}(\mathbf{x} + c)$.

b) The `Softmax` node in `edf.py` subtracts the largest score $\max_k x_k$ from every score before it applies the exponential function. Why does this not change the result, and why is it useful?

1.3 1-layer MLP

Consider a 1-layer neural network $\mathbf{y} = g(\mathbf{AX})$ with input $\mathbf{X}$, output $\mathbf{y}$, network weight $\mathbf{A}$ and output function g. The network is applied to a minibatch of three 2D input points, which are the columns of $\mathbf{X}$. Let's first assume the input and the network weights are

$$\mathbf{A} = \begin{pmatrix} 3.0 & 2.0 \end{pmatrix} \in \mathbb{R}^{1\times 2} \qquad \mathbf{X} = \begin{pmatrix} 2.0 & 1.0 & -1.0 \\ 4.0 & -2.0 & 0.0 \end{pmatrix} \in \mathbb{R}^{2\times 3}$$

a) Assume the function g is linear, i.e. $g(\mathbf{AX}) = \mathbf{AX}$. Given the target output $\mathbf{t}$ and the loss function $\mathcal{L}$, perform one gradient descent update of the weights with learning rate $\eta = 1$. As in the lecture, average the gradient over the minibatch: $\mathbf{A} \leftarrow \mathbf{A} - \eta \frac{1}{3} \frac{\partial \mathcal{L}}{\partial \mathbf{A}}$. Please provide the loss *before* and *after* the weight update.

$$\mathbf{t} = \begin{pmatrix} 15 & 3 & 1 \end{pmatrix} \in \mathbb{R}^{1\times 3} \qquad \mathcal{L} = \frac{1}{2} \sum_j (y_j - t_j)^2$$

Hint: First calculate the forward pass and obtain the loss. Next, derive the gradients of the loss wrt. every element of the network weight, $\frac{\partial \mathcal{L}}{\partial a_i}$. Make sure to include all steps of your derivation. Once the network weight is updated, calculate the forward pass again to obtain the new loss.

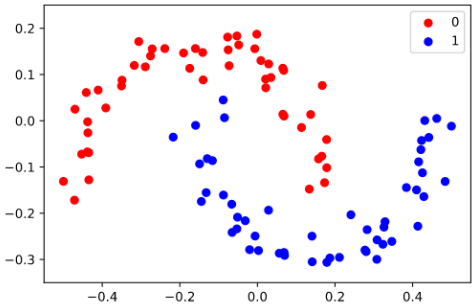
b) For the loss that you calculated before and after the weight update in **a)**, does the loss decrease, or in other words, is the new prediction closer to the target $\mathbf{t}$? If not, what could be the reason?

c) Now we set the function g to be ReLU, applied to every element of $\mathbf{AX} \in \mathbb{R}^{1\times 3}$. Use the original weights $\mathbf{A}$ and the same target $\mathbf{t}$ and loss $\mathcal{L}$ as in **a)**. Calculate the gradients wrt. the network weight, $\frac{\partial \mathcal{L}}{\partial a_i}$. You can use the chain rule $\frac{\partial \mathcal{L}}{\partial a_i} = \frac{\partial \mathcal{L}}{\partial g} \frac{\partial g}{\partial a_i}$. What do you notice in the calculated gradient?

Hint: Look at the sign of each element of $\mathbf{AX}$ and use the derivative of ReLU from 1.1 a) with $c = 0$.

2 Coding

2.1 Binary Classification on 2D Point Cloud



First of all, you can directly use your conda environment installed for exercise 1. You can simply run `conda activate deep_learning_ex_1` in your terminal. If you don't have the environment, please check `install_guide.txt` in exercise 1.
Now, in the jupyter notebook `pcl_binary_classification.ipynb`, you can find the code for loading a 2D point cloud dataset. The training code is also provided. Here, the input `X` contain the 100 2D points, and `y` are their corresponding labels (0 or 1). The goal is to train a model that can classify every point to its correct label.

a) As the first task, you are required to train a logistic regression model with help of the Educational Framework (EDF). The EDF's `CrossEntropyLoss` should be used. You can also reuse your code from exercise 1.

b) Next, visualize your trained model. What we do here is, within the range of [-0.5, 0.5] in both x and y axis, we sample points with the space of 0.002. In this way, there are $500 \times 500 = 250K$ 2D points in total, denoted as `X_plot`. We want to predict their corresponding label with your trained logistic regression model. To this end, we have built the input as `input = edf.Parameter(X_plot)`. All you need to do is to fill `output` with your trained model. If everything is correct, you should see a decision boundary in black.

Hint: you can basically use your code in assembling the computational graph, but remove `SingleProbToProbVector`.

c) After visualizing the classification result, you should notice that your logistic regression model is not able to classify the points correctly: its decision boundary is a straight line, but the two classes cannot be separated by a straight line. An MLP can learn a non-linear decision boundary. Therefore, now your task is to implement your first Multi-Layer Perceptron (MLP) for the same binary classification task. You should implement an MLP with two layers: the first one takes the input point coordinates and the other one outputs the classification predictions. The hidden dimension is set to `nHiddens`=24. For example, given 100 points as input, the first affine node should output a matrix of 100×24. `Sigmoid` is used as the activation function in the hidden layer as well as the final output function.

d) Similar to **b)**, visualize and verify your trained MLP model. Take the `input`, you should fill `output` with your model. You should be able to see that MLP is able to classify the 2D point clouds properly with a bending decision boundary.

2.2 Image Classification on MNIST

We can now move to `image_classification.ipynb`. Unlike in exercise 1 where we only pick two digits to perform binary classification, now we move to a slightly more difficult task: classification on all 10 digits.



There are $60K$ `train_images` and $10K$ `test_images`, where every image is a 784-dim vector reshaped from the original grayscale image with the resolution of 28×28. Correspondingly, `train_labels` and `test_labels` contain their real labels between 0 and 9. The goal of this homework is to train a model that can make accurate classification given some hand-written digit images.

a) Again, the first task is to implement our old friend, the logistic regression model. You should be able to reuse your code from before. Note that the output dimension should be `nOutputs`=10 now, which represents the predicted probability on all 10 classes. Also, instead of using `Sigmoid` we replace the output function now with `Softmax`. You can directly use the implemented `Softmax` in `edf.py`. Moreover, we don't need `SingleProbToProbVector` any more. Finally, you should use `CrossEntropyLoss` as your loss function. If the model is correctly implemented, the test error should be below 9% after 10 epochs.

b) *Open Question*: If your logistic regression model weight is randomly initialized, and no training is performed, what test error do you think the model will get? Please answer inside the notebook.

c) Now you should implement an MLP with two linear layers and `Sigmoid` as the activation function in the hidden layer. Note that you should also use `Softmax` as your function for the final output. A large part of your code in 2.1 can be reused. If it is correctly implemented, the test error should now be below 4% after 10 epochs.

d) Besides `Sigmoid`, there are many other activation functions. We want to explore the different performance when using different activation functions. Your task now is to implement the `forward` and `backward` pass for the new activation functions. The basic structure has been provided in the notebook. Note that you can still base your implementation on `Sigmoid` in `edf.py`. Moreover, the analytical derivatives that you derived in 1.1 are also useful. For each activation function, we also provide the code to plot output and analytical gradients computed by your implementation as a sanity check.

- **i)** Implement `Tanh`.
- **ii)** Implement `ReLU`.
- **iii)** Implement `SiLU`, $\text{SiLU}(x) = x\,\sigma(x)$. You can reuse the sigmoid code of `edf.py`.

After you implement every new activation function, you simply need to define a computation graph and train your MLP with this function. Follow the instructions inside the notebook. If correctly implemented, the test error should be below 4% after 10 epochs for all of them.

e) After implementing all the activation functions, now we want to check the analytical gradients with numerical differentiation. Given an activation function f and an input vector $\mathbf{x}$, please implement the numeric gradient of f using symmetric difference quotient, i.e. $\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \approx \frac{f(\mathbf{x}+h)-f(\mathbf{x}-h)}{2h}$, where $h = 10^{-4}$. You can consider $\mathbf{x}+h$ and $\mathbf{x}-h$ as objects of `edf.Input()`. Then, you perform a forward pass and calculate `numeric_grad`. Once you are done, you can use the provided plot code to verify. The curve for analytical gradient and numeric gradient should overlap each other.

f) Let's check how changing the learning rate affects the model's train and test error. We provide the for-loop over the provided list `learning_rates` = $[0.1, 0.5, 1.0]$ and use a new learning rate every time `edf.learning_rate = learning_rates[i]`. You should implement the training process in a way to collect the network's final train/test errors for each of the learning rates, and keep them in the lists `train_err_per_lr` and `test_err_per_lr`. Try to find the best learning rate for both `ReLU` and `Sigmoid`. Note that you can take the training code in **c)** or **d)** as reference.

g) We will check how the number of hidden layers affects the model's performance. Similar to **f)**, you will see how the depth of the network `nLayers` affects its performance. We provide a list `num_layers` = $[2, 4, 6]$. Therefore, you can use a for-loop over the list `num_layers`, and you update your intermediate output within the loop. First try with the Sigmoid MLP and then a ReLU MLP. Again, populate `train_err_per_depth` and `test_err_per_depth` accordingly. Your code should support multiple hidden layers (note the new nLayers argument). Make sure that each hidden layer in your MLP should have the same `nHiddens` number of neurons.

h) Next, let's check how changing hidden dimensions in your MLP affect the model's performance. Change `nHiddens` within a for-loop. Again, populate `train_err_per_hidden` and `test_err_per_hidden` accordingly. You can reuse most of your code from above.

i) *Open Question*: The figure below shows the final train and test errors of our ReLU MLP with two hidden layers for different hidden dimensions. With 128 hidden units, the network has about 118K parameters, about twice as many as there are training images. Would you expect such a model to overfit, i.e. to reach a low train error but a high test error? What do you observe in the figure and in your own results? Please write down your answer in the provided cell in the notebook.

